

Linear Search

Check one position at a time and return the first matching index; report -1 only after every position has failed.

This complete offline chapter connects ideas, diagrams, exact traces, commented Java and exam practice. Switch themes for reading; printing always uses the light presentation and includes every required trace and answer.

GUIDE 2026-10-07.1 · STANDALONE HTML AND PDF SHARE THE SAME CONTENT

1. Meet the array and the goal

An array stores values at zero-based indexes. A search key is the value you want; the answer is its position or a failure marker.

63	29	85	72	18	42	23	54	8	32
0	1	2	3	4	5	6	7	8	9

Index labels belong to positions, not to the values. Find key 18.

The last valid index is 9. Returning 18 would confuse a value with an index.

2. Definition and plain-language idea

Linear search, also called sequential search, compares each position with the key in order.

63	29	85	72	18	42	23	54	8	32
0	1	2	3	4	5	6	7	8	9

first check

Begin at index 0; all later positions remain possible until checked.

Check one position at a time and return the first matching index; report -1 only after every position has failed.

Check index 0
63 == 18 → false



Check index 1
29 == 18 → false



Continue until
match or end

Each failed equality test advances exactly one index; the full array need not be sorted.

No sorted-order precondition. Equality provides no reason to skip distant positions.

3. Vocabulary before the trace

Term	Meaning in this chapter
Key	Target value, compared with arr[i].
i	The current index, from 0 to length - 1.
Match	arr[i] == key for primitive int values.
Sentinel -1	Signals not found; it is never a valid array index.
First occurrence	The earliest matching position because the scan returns immediately.

4. Visual trace: predict, then advance

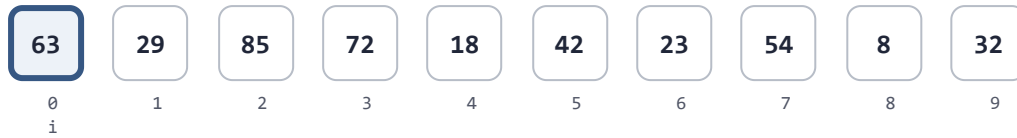
Before pressing Next, say which position will be inspected and what the bounds/index will become. The marked row and picture share one selected step.

Search for 18

▼ Full trace table - select a row to jump to its scene

Step	i	arr[i]	Evaluated condition and result
1	0	63	63 == 18 → false; advance i
2	1	29	29 == 18 → false; advance i
3	2	85	85 == 18 → false; advance i
4	3	72	72 == 18 → false; advance i
5	4	18	18 == 18 → true; return 4

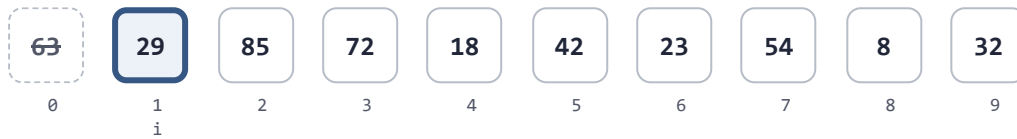
Check 1: index 0



Key 18; checks proceed left to right.

63 == 18 → false; advance i

Check 2: index 1



Key 18; checks proceed left to right.

29 == 18 → false; advance i

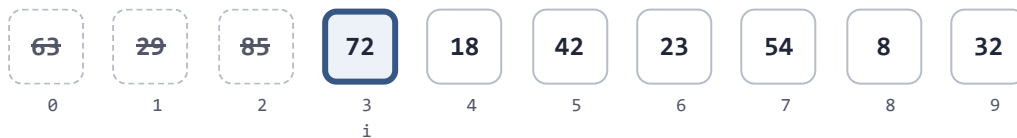
Check 3: index 2



Key 18; checks proceed left to right.

85 == 18 → false; advance i

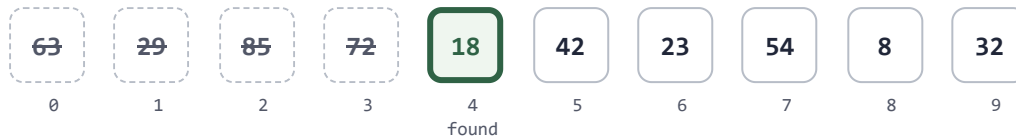
Check 4: index 3



Key 18; checks proceed left to right.

72 == 18 → false; advance i

Check 5: index 4



Key 18; checks proceed left to right.

18 == 18 → true; return 4

Return 4 after 5 comparisons.

5. Why the movement is valid

Before index i is checked, every position $0..i-1$ has failed. After a failure, advance by exactly one position. On a match, the invariant proves this is the first match.



Searching for 7 returns 0 even though index 2 also matches.

6. Fully commented Java implementation

JAVA

```
public static int linearSearch(int[] arr, int key) {  
    // Every valid index must be checked until a match is found.  
    for (int i = 0; i < arr.length; i++) {  
        // Return the INDEX, not the value stored there.  
        if (arr[i] == key) return i;  
    }  
    // Only report failure after all valid indexes were checked.  
    return -1;  
}
```

Build it from scratch

- Accept an array and key.
- Visit every valid index from 0.

- Return the matching index immediately.
- Put return -1 after the loop.

7. Recursive form and the call stack

JAVA

```
public static int linearSearchRec(int[] arr, int key, int index) {
    // Empty remaining suffix: do not access arr[index].
    if (index >= arr.length) return -1;
    // Equality finds the first matching position from this index.
    if (arr[index] == key) return index;
    // Start the first call at index 0; each call shrinks the suffix.
    return linearSearchRec(arr, key, index + 1);
}
```

CALL STACK - NEWEST FRAME AT THE TOP

linearSearchRec(index 2) Check the current suffix
linearSearchRec(index 1) Waiting: Return child result
linearSearchRec(index 0) Waiting: Return child result

Each failure suspends a parent and searches a smaller suffix. Ordinary Java does not eliminate this recursion stack.

Initial call: `linearSearchRec(arr, key, 0)`. Worst-case depth is $O(n)$; the iterative version is preferable when that stack cost matters.

8. Complete runnable program

Save as `LinearDemo.java`, compile with `javac`, then run the class name without an extension.

JAVA

```
import java.util.Arrays;

public class LinearDemo {
    public static int linearSearch(int[] arr, int key) {
        // Every valid index must be checked until a match is found.
        for (int i = 0; i < arr.length; i++) {
            // Return the INDEX, not the value stored there.
            if (arr[i] == key) return i;
        }
    }
}
```

```

    // Only report failure after all valid indexes were checked.
    return -1;
}

public static void main(String[] args) {
    int[] values = {63, 29, 85, 72, 18, 42, 23, 54, 8, 32};
    System.out.println(linearSearch(values, 18));
}
}

```

EXPECTED OUTPUT

4

9. Five worked cases and exact trace tables

1. Absent key

Input [63, 29, 85, 72, 18, 42, 23, 54, 8, 32]; key 31.

63	29	85	72	18	42	23	54	8	32
0	1	2	3	4	5	6	7	8	9

Final probe/check at index 9; return -1.

Check	i	arr[i]	Evaluated condition and result
1	0	63	63 == 31 → false; advance i
2	1	29	29 == 31 → false; advance i
3	2	85	85 == 31 → false; advance i
4	3	72	72 == 31 → false; advance i
5	4	18	18 == 31 → false; advance i
6	5	42	42 == 31 → false; advance i
7	6	23	23 == 31 → false; advance i
8	7	54	54 == 31 → false; advance i
9	8	8	8 == 31 → false; advance i
10	9	32	32 == 31 → false; advance i

Result -1; 10 key comparisons.

2. First match among duplicates

Input [7, 3, 7, 1]; key 7.

7371

0123

Final probe/check at index 0; return 0.

Check	i	arr[i]	Evaluated condition and result
1	0	7	7 == 7 → true; return 0

Result 0; 1 key comparisons.

3. Match at the last index

Input [9, 4, 2, 18]; key 18.

94218

0123

Final probe/check at index 3; return 3.

Check	i	arr[i]	Evaluated condition and result
1	0	9	9 == 18 → false; advance i
2	1	4	4 == 18 → false; advance i
3	2	2	2 == 18 → false; advance i
4	3	18	18 == 18 → true; return 3

Result 3; 4 key comparisons.

4. Negative values

Input [-5, -2, 0, 8]; key -2.



Final probe/check at index 1; return 1.

Check	i	arr[i]	Evaluated condition and result
1	0	-5	-5 == -2 → false; advance i
2	1	-2	-2 == -2 → true; return 1

Result 1; 2 key comparisons.

5. Empty and singleton

Input [18]; key 31.



Final probe/check at index 0; return -1.

Check	i	arr[i]	Evaluated condition and result
1	0	18	18 == 31 → false; advance i

Result -1; 1 key comparisons. For [] the loop/body is never entered and -1 is returned with zero probes.

10. Variations: order, duplicates and reference values

To find the last occurrence, continue scanning and retain the latest matching index. To count matches, increment a counter instead of returning. Descending, ascending and unsorted arrays all use the same equality scan.

JAVA

```
public static int findName(String[] names, String key) {
    for (int i = 0; i < names.length; i++) {
        // Unfilled reference cells are null; && protects the method call.
        // equals compares text; == would compare object references.
        if (names[i] != null && names[i].equals(key)) return i;
    }
    return -1;
}
```

Objects need a field comparison or an appropriate equals method. Guard nullable references before invoking a method.

11. Properties and why the cost occurs

Property	Result	Reason
Best time	$O(1)$	First position matches.
Average / worst	$O(n)$	Number of visited positions grows with input length.
Extra space	$O(1)$ iterative; $O(n)$ recursive	Index versus suspended frames.
Input order	Not required	Checks use equality only.
Duplicates	First match	Immediate return.
Input mutated?	No	The method only reads cells.

First value		1 comparison
First half		About $n / 2$ comparisons
Last or absent		n comparisons

Linear growth: doubling the number of candidates can double the scan.

The tables count one primitive-key equality test per visited cell. Boundary/loop conditions are not included.

12. When to choose this method

Choose linear search for small, unsorted, or one-off searches. It avoids sorting cost and works with sequential access structures.

13. Common mistakes and repairs

Mistake	Consequence	Repair
Use <code>i <= arr.length</code>	Reads one cell beyond the array.	Use <code>i < arr.length</code> .
Return -1 inside the loop after first failure	Stops before later matches.	Return failure after the loop.
Return <code>arr[i]</code>	Returns a value rather than position.	Return <code>i</code> .
Use <code>==</code> for String text	Compares references.	Use a null-safe equals test.

14. Exam-style practice with explanations

A. Trace key 31 on the main input. Show every index/bound and the failure result. [6]

Check	i	arr[i]	Evaluated condition and result
1	0	63	<code>63 == 31</code> → false; advance i
2	1	29	<code>29 == 31</code> → false; advance i
3	2	85	<code>85 == 31</code> → false; advance i
4	3	72	<code>72 == 31</code> → false; advance i
5	4	18	<code>18 == 31</code> → false; advance i
6	5	42	<code>42 == 31</code> → false; advance i
7	6	23	<code>23 == 31</code> → false; advance i
8	7	54	<code>54 == 31</code> → false; advance i
9	8	8	<code>8 == 31</code> → false; advance i
10	9	32	<code>32 == 31</code> → false; advance i

Every retained candidate is eliminated; return -1. Ten equality checks are required.

B. Explain the best and worst time using input examples. [4]

Best: key at index 0, one equality test. Worst: key last or absent, n equality tests.

C. Debug the bounds and return statement. [4]

i must stay below length; a failed equality test means continue, not return -1. Failure belongs after the loop.

D. Return the first occurrence of a duplicate. What must change? [3]

Nothing: the left-to-right immediate return already selects the first occurrence.

E. Empty array: can this code access index 0? [2]

No. $i = 0$ is not less than $\text{length} = 0$. It returns -1 without reading a cell.

15. Test-ready summary and mastery check

Check one position at a time and return the first matching index; report -1 only after every position has failed.

- Return an index, or -1 after the full scan.
- Keep every index in $0..\text{length}-1$.
- Know why unsorted input is acceptable.
- Explain first-match behavior.
- Use equals for reference text.

CSI247 visual study guide · Linear Search · 2026-10-07.1

Structural reference: the author-supplied CSI247 guide, with topic-specific adaptation and repo-owned teaching content. All traces in this artifact are generated by deterministic simulations of its stated procedures. Counts distinguish key comparisons from loop tests, probes, swaps, shifts and array writes.